2026-2027 学年第一学期

《高级语言程序设计(C++语言)》

实验报告

实验名称：第一次实验课	
实验日期：2026.10.23	实验地点：于刚宋晓楼 A237
学　　号：2026312192018	姓　　名：蔡旻晧
学　　院：人工智能	专　　业：人工智能
指导老师：丁建利	

一、实验目的

1. 熟悉 C++ 源程序的基本结构，理解头文件包含、命名空间、主函数与返回值的作用。
2. 掌握 cin / cout 与 scanf / printf 两组输入输出语句的用法，能正确控制输入输出的格式。
3. 掌握顺序结构、选择结构（if、if-else、if-else if 多分支）和循环结构（for、while）的书写方法及其嵌套使用。
4. 学会用枚举、迭代、累加、整数拆分、字符串处理等基本方法分析并解决实际问题。
5. 学会用样例数据验证程序的正确性，能够定位并改正逻辑错误与边界处理错误，养成规范的缩进和注释习惯。

二、实验环境

1. 硬件环境：个人计算机（Intel Core i5 处理器，16 GB 内存）。
2. 操作系统：Windows 11（64 位）。
3. 开发工具：Dev-C++ 5.11 / Visual Studio Code，编译器为 MinGW-w64 中的 g++。
4. 编译标准：C++17，编译命令 g++ 源文件.cpp -o 可执行文件.exe。

5. 调试方式：修改—编译—运行循环，配合样例数据手工验算和断点单步调试。

三、实验过程及结果

本次实验共 8 道程序题，全部编译、运行通过，运行结果见各题截图。下面按题目分别给出题目要求、算法思路、源代码、运行结果和说明。

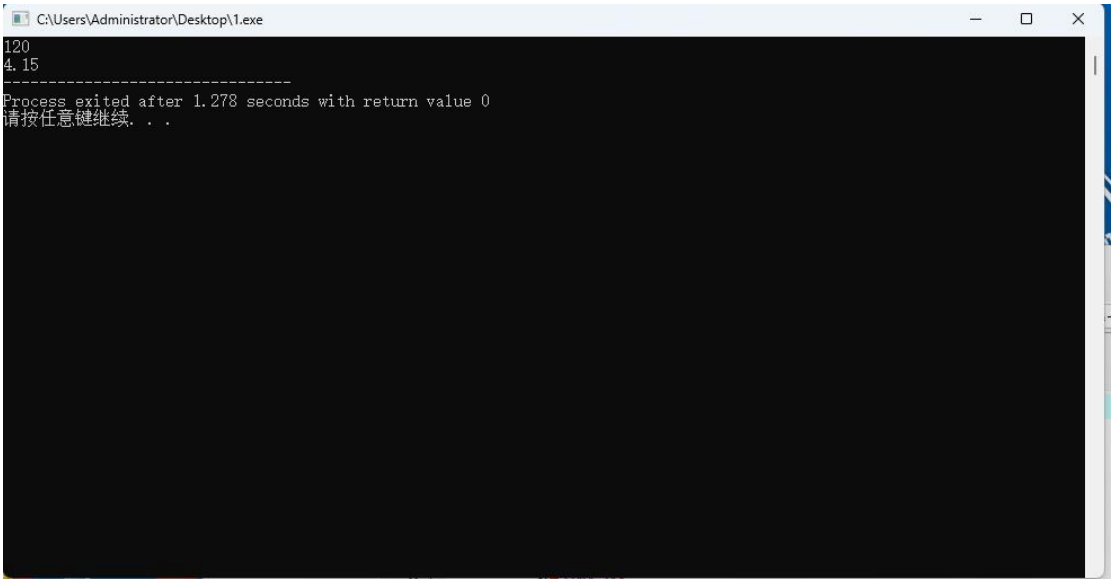
第一题　按利润分段计算奖金

题目：企业发放的奖金根据利润分段提成：利润不超过 10 万元的部分按 10% 提成；10～20 万元的部分按 7.5%；20～40 万元的部分按 5%；40～60 万元的部分按 3%；60～100 万元的部分按 1.5%；超过 100 万元的部分按 1% 提成。输入利润（单位：万元），输出应发放的奖金（单位：万元）。

算法思路：从高到低依次判断利润落在哪一档。每处理完一档，就把已经计算过的部分从利润中截掉（把 n 改成该档的上限），这样后面的判断只需要处理剩下的部分，最后把所有档的提成累加起来即可。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    float n,s=0;
    cin>>n;
    if(n>100){
        s+=(n-100)*0.01;
        n=100.0;
    }
    if(60<n){
        s+=(n-60.0)*0.015;
        n=60;
    }
    if(40<n){
        s+=(n-40.0)*0.03;
        n=40;
    }
    if(20<n){
        s+=(n-20.0)*0.05;
        n=20;
    }
    if(10<n){
        s+=(n-10.0)*0.075;
        n=10;
    }
    if(0<n<=10){
```

```
            s+=n*0.1;
            n=0;
        }
        cout<<s;
        return 0;
    }
```



运行结果：输入 120，输出 4.15；见运行截图。

说明：最初把最后一档的条件写成了 0<n<=10。C++ 中关系运算符从左向右结合，它等价于 (0<n)<=10，其中 (0<n) 得到 0 或 1，永远小于等于 10，因此该条件恒为真，相当于取消了这一档的判断。由于前面的 if 已经把 n 截断到 10，样例结果恰好正确，问题很容易被忽略；这一档的条件应写成 n>0 才与题意一致。

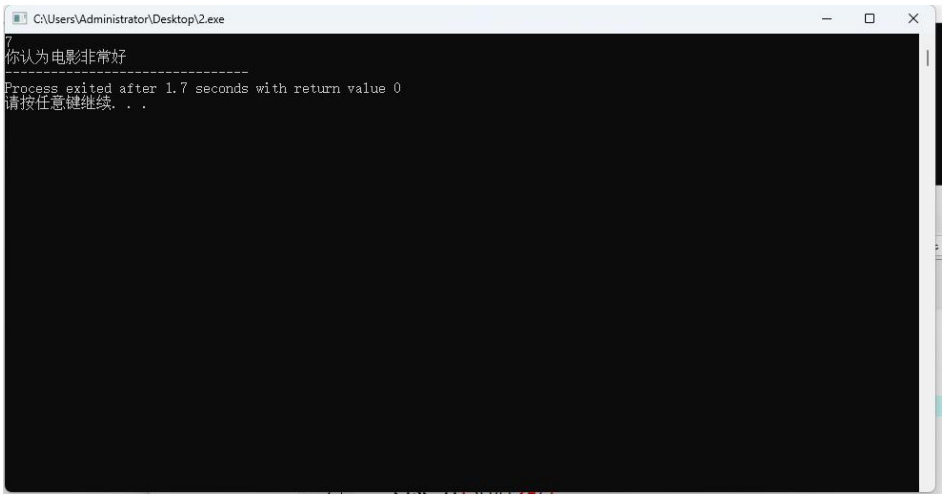
第二题　根据评分输出影评

题目：输入一个 0～10 的整数评分，输出对应的评价：9 分及以上为“经典”，7～8 分为“非常好”，5～6 分为“一般”，5 分以下为“烂片”。

算法思路：用 if-else if 多分支结构，按分数从高到低判断：第一个成立的分支就是结果，因此每个分支只需判断下界，不必再写上界。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int score;
    scanf("%d",&score);
    printf("你认为电影");
    if(score>=9)
        printf("经典");
    else if(score>=7)
        printf("非常好");
    else if(score>=5)
        printf("一般");
```

```
        else printf("烂片");
        return 0;
    }
```



运行结果：输入 7，输出“你认为电影非常好”；见运行截图。

说明：多分支的判断顺序很关键，如果把 score>=5 写在 score>=7 的前面，8 分也会被判成“一般”，所以条件必须按由严到宽的顺序排列。另外程序里 scanf/printf 与 cin 混用，输出风格不统一，且末尾没有换行，多次运行的输出会连在一行，建议统一并在最后补一个换行。

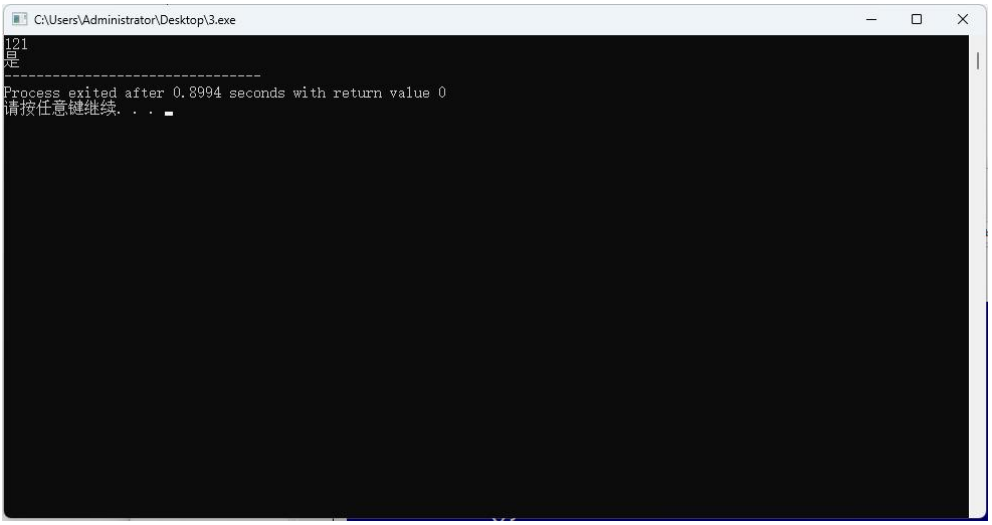
第三题　判断回文数

题目：输入一个整数，判断它是否为回文数（正着读和倒着读相同，例如 12321），是则输出“是”，否则输出“不是”。

算法思路：先用 stringstream 把整数转换成字符串，再用两个下标从首、尾同时向中间比较；只要有一对字符不相同就把标志位置 1 并提前结束循环，最后根据标志位输出结果。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n,flag=0,len;
    scanf("%d",&n);
    string s;
    stringstream ss;
    ss<<n;
    ss>>s;
    len=s.size();
    for(int i=0;i<=int(len/2);i++){
        if(s[i]!=s[len-i-1]){
            flag=1;
            break;
        }
    }
```

```
        if(flag==0)printf("是");
        else printf("不是");
        return 0;
    }
```



运行结果：输入 121，输出“是”；见运行截图。

说明：如果直接对整数反复取余、除以 10 再比较，还要额外处理位数和末尾的 0，容易出错；转成字符串后按下标首尾比较，逻辑直观得多。循环条件写成 i<=len/2 与 i<len/2 结果相同，因为对称位置比较到中间即可。

第四题　求 100～999 之间的水仙花数

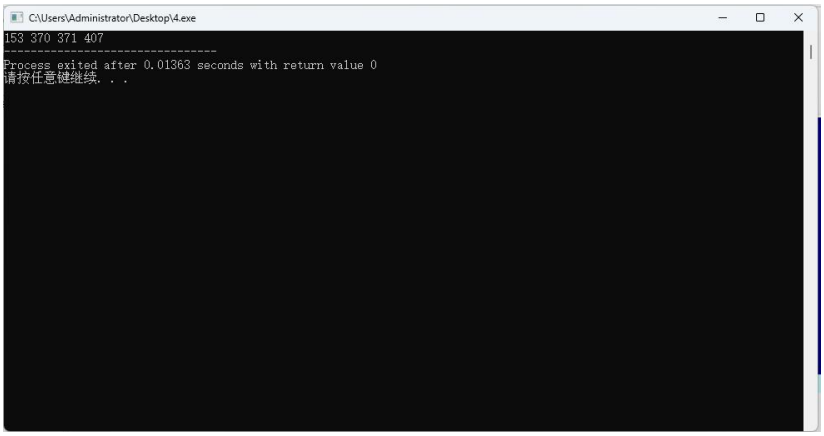
题目：输出 100～999 之间所有的水仙花数，即各位数字的立方和等于该数本身，例如 $153 = 1^3 + 5^3 + 3^3$。

算法思路：枚举 100～999 的每一个整数，用 /100、/10%10、%10 分别取出百位、十位、个位，判断三个数字的立方和是否等于原数，相等则输出。

```
#include<bits/stdc++.h>
using namespace std;
int main(){

    for(int i=100;i<=999;i++){
        int a,b,c;
        int s=0;
        a=int(i/100);
        b=int(i/10)%10;
        c=i%10;
        s+=a*a*a+b*b*b+c*c*c;
        if(s==i)printf("%d ",i);
    }
    return 0;

}
```



运行结果：输出 153 370 371 407；见运行截图。

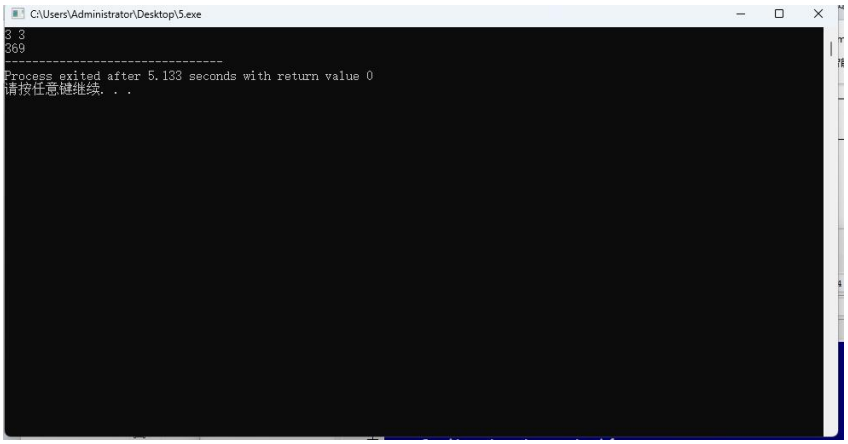
说明：用整数除法和取余拆分各位数字是最常用的方法；枚举范围固定，循环只有 900 次，无需考虑效率优化。

第五题　求 a+aa+aaa+… 的和

题目：输入 n 和 a，求 a＋aa＋aaa＋…（共 n 项，最后一项由 n 个 a 组成）的值。例如 n＝5、a＝2 时，和为 2＋22＋222＋2222＋22222＝24690。

算法思路：直接构造 a、aa、aaa 并不方便，程序利用等价变换：先求 1、11、111…… 的累加和（s 初值为 1 计入第一项，之后每轮先把 b 变成下一项 b=b*10+1 再累加，共 n 项），最后把 s 整体乘以 a，即得 a＋aa＋aaa＋… 的和。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n,a,s=1,b=1;
    cin>>n>>a;
    for(int i=0;i<n-1;i++){
        b=b*10+1;
        s+=b;
    }
    cout<<s*a;
    return 0;
}
```



运行结果：输入 3 3，输出 369，即 3＋33＋333；见运行截图。

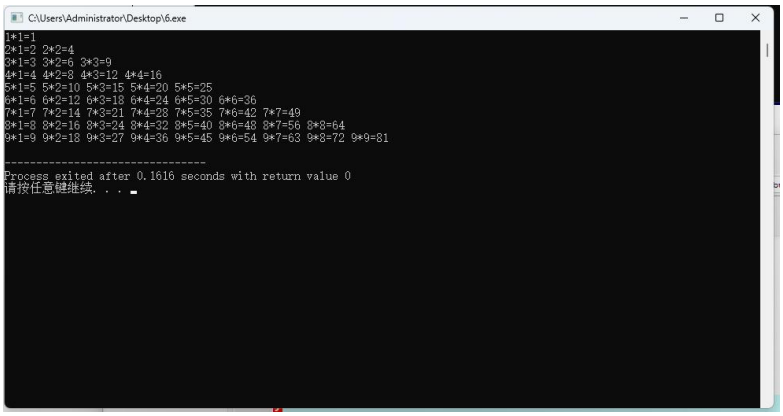
说明：本题的关键是等价变换：把一个“不容易构造”的数列换成“容易递推”的 1、11、111…… 再整体缩放，代码短且不容易出错。

第六题　输出九九乘法表

题目：按下三角形式输出九九乘法表，共 9 行，第 i 行有 i 个算式。

算法思路：外层循环 i 控制行（1～9），内层循环 j 控制列（1～i），输出算式后由内层循环外的 printf("\n") 换行。内层循环的上界随外层变量变化，这是典型的“循环次数依赖外层变量”的嵌套结构。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    for(int i=1;i<=9;i++){
        for(int j=1;j<=i;j++)
            printf("%d*%d=%d ",i,j,i*j);
        printf("\n");
    }
    return 0;
}
```



运行结果：输出下三角九九乘法表，第 1 行 1*1=1，第 9 行 1*9=1 …… 9*9=81；见运行截图。

说明：换行的 \n 必须放在内层循环之外，放进去就变成每行只输出一个算

式了。另外本程序输出的是 i 在前的 i*j，与常见的 j 在前写法（j*i）乘积相同但顺序不一致。

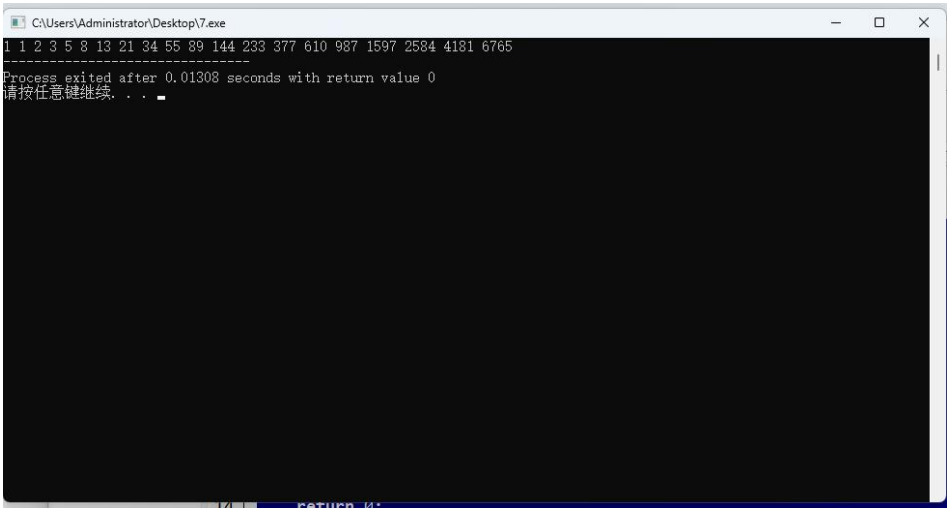
第七题　输出斐波那契数列前 20 项

题目：输出斐波那契数列的前 20 项。数列满足 F1 = F2 = 1，从第三项起 Fn = Fn-1 + Fn-2。

算法思路：用 a、b 两个变量保存相邻的两项，先输出前两项，然后循环 18 次：c = a + b 得到新的一项并输出，再把 a、b 各向后移动一位。只需常数个变量，不必使用数组。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int a=1,b=1,c,tmp;
    printf("%d %d ",a,b);
    for(int i=0;i<18;i++){

        c=a+b;
        tmp=c;
        a=b;
        b=tmp;
        printf("%d ",c);
    }
    return 0;
}
```



运行结果：输出 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765；见运行截图。

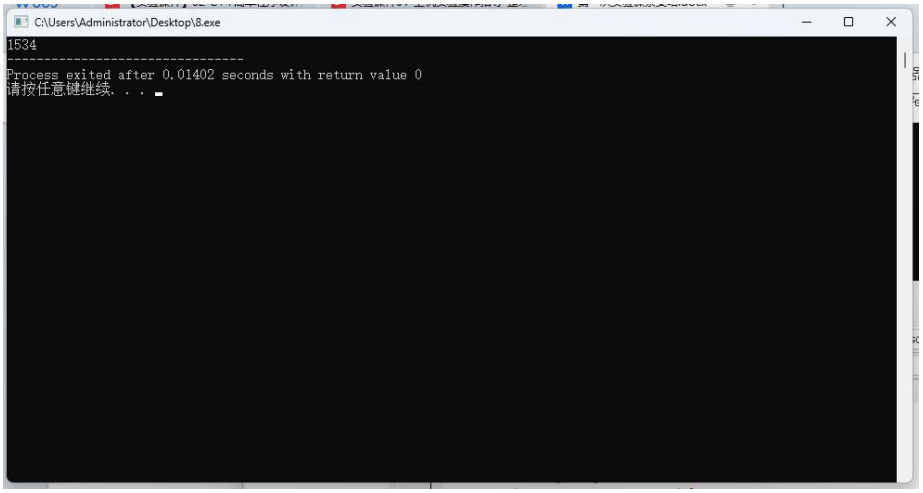
说明：程序里用临时变量 tmp 做交换，多引入了一个不必要的变量，直接 a = b; b = c; 即可。项数继续增多时建议把整型换成 long long，防止溢出。

第八题　猴子吃桃问题

题目：猴子第一天摘下若干个桃子，当天吃掉一半多一个；以后每天早上都吃掉前一天剩下的一半多一个；到第 10 天早上想吃时，发现只剩下 1 个桃子。求第一天共摘了多少个桃子。

算法思路：正向推算比较困难，改用逆推：第 10 天早上剩下 s=1 个，那么前一天的数量为 (s+1)*2。从第 9 天倒推到第 1 天共 9 次，得到第一天摘下的桃子数。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int n=10,s=1;
    for(int i=0;i<n-1;i++){
        s=(s+1)*2;
    }
    cout<<s;
    return 0;
}
```



运行结果：输出 1534；见运行截图。

说明：逆推时“吃掉一半多一个”要反过来变成“先加 1 再乘 2”；第 10 天剩下的 1 个是已知的初始条件，不参与逆推，所以循环 9 次而不是 10 次。这类“差一次”的问题，用第 1、2 天的小数据手工验算一遍最不容易错。

四、实验总结（本次实验的收获、体会、经验和教训）

4.1 主要收获

1. 通过 8 道题目把顺序、选择、循环三种基本控制结构都实际用了一遍，对 if-else if 多分支和双层 for 循环的嵌套有了更直观的认识。
2. 总结出几个常用套路：分段计算要从高到低判断并截断已处理的部分；拆分整数用 / 和 %；数列题先找递推关系或做等价变换；逆推问题先确定最后一天的状态再倒推。
3. 学会了用 stringstream 在整数与字符串之间转换，比手写取余、除 10 的循环

更简洁。

4.2 遇到的问题及解决

1. 第一题把最后一档的条件写成 0<n<=10。C++ 的关系运算符左结合，(0<n)<=10 恒为真，相当于取消了这个分支的判断；由于前面的 if 已经把 n 截断到 10，样例结果恰好正确，很容易被忽略。这一档应写成 n>0——结果正确并不等于逻辑正确。
2. 第二题混用了 scanf/printf 和 cin/cout，输出风格不统一，而且末尾没有换行，多次运行的结果会连在同一行；建议统一成一种输入输出方式，并在末尾补上换行。
3. 第六题输出的算式是 i 在前（i*j），虽然乘积相同，但与常见的乘法表写法（j 在前）不一致，建议调整为 j*i。
4. 第七题用了一个额外的临时变量 tmp 做交换，多引入了一个不必要的变量，可以简化为 a = b; b = c;。
5. 第八题逆推时循环次数容易“差一次”，一开始不确定该循环 9 次还是 10 次；用第 1、2 天的小数据手工推了一遍，才确认第 10 天剩下的 1 个不参与逆推。

4.3 体会与改进

1. 编译通过、样例通过只是最低要求，边界数据（0、负数、个位数、最大值）才是最容易暴露问题的地方，以后每道题都要专门设计几组边界数据来测试。
2. 代码风格要统一：缩进、变量命名、必要的注释。这次整理报告时深有体会——几天前写的代码如果没有注释、缩进混乱，回头看要重新读一遍才知道在做什么。
3. 下一步打算继续练习函数与数组，把这次重复出现的代码（例如拆分各位数字）封装成函数，提高代码的复用性；同时尝试用调试器单步跟踪循环，而不是只靠输出中间结果来定位问题。